

General Sir John Kotelawala

Defence University

Faculty of Computing

Artificial Intelligence Final Report

Group Details		
Student Name	**Registration Number**	**Degree Program**
KAUC Prabhashwara	D/BIS/24/0004	Information System
TMND Hemarathne	D/BIT/24/0070	Information Technology
DAKHD Arachchi	D/BSE/25/0009	Software Engineering
PADCM Perera	D/DBA/25/0018	Data Science & Business Analytics
Project Title: Serendib Go – AI-Based Intelligent Online Bus Booking Management System		

Table of Contents

1. Abstract iv
2. Introduction vi
3. Problem Background viii
4. Problem Statement x
5. Objectives of the Proposed AI Solution xi
Objective 1 - Segment-Based Demand Prediction xi
Objective 2 - Traveler Pattern Clustering xi
Objective 3 - Segment Seat Reselling and Inventory Decoupling xi
Objective 4 - Local AI Travel Assistance xi
Objective 5 - Full-Stack System Integration xii
6. Selected AI Techniques xiii
6.1 Supervised Machine Learning: XGBoost Regression xiii
6.2 Unsupervised Machine Learning: K-Means Clustering xiii
6.3 Local Generative AI: Qwen 2.5 (qwen2.5:3b via Ollama) via Ollama xiv
7. Justification of AI Techniques xvi
7.1 XGBoost Compared with Deep Neural Networks and Linear Regression xvi
7.2 K-Means Compared with Hierarchical Clustering xvi
7.3 Local Ollama and Qwen 2.5 (qwen2.5:3b via Ollama) Compared with Cloud AI APIs xvii
8. System Architecture & Design xix
Tier 1: Frontend Application Interface (Next.js 16 + React 19) xix
Tier 2: Business Logic and API Gateway (Laravel 12) xx
Tier 3: AI and Machine Service Engine (FastAPI + Local Ollama) xx
9. Technical Implementation Details & Code Proofs xxi
9.1 Database Schema Design (MySQL 8.0) xxi
9.2 SQL Interval Tree Overlap Query Engine & Indexing Guidance xxi
9.3 Production FastAPI Machine Learning Microservice (predict_demand.py) xxii
9.4 Local Ollama Terminal Verification Log xxiii
10. Testing, Simulations & Results xxiv
11. Limitations & Critical Discussion xxvi
11.1 Physical Boarding and Passenger Overstaying xxvi
11.2 Traffic Delays and Schedule Changes xxvi
11.3 Internet Connectivity Problems xxvi
12. Ethical, Social & Privacy Considerations xxvii
12.1 Privacy-Focused Architecture xxvii

12.2 Fair Fare Allocation and Anti-Surge Pricing xxvii
12.3 Digital Inclusivity and Accessibility.....xxvii
13. Conclusion & Future Work..... xxviii
xxviii
14. References.....xxix
15.Appendix.....xxix

1. Abstract

Inter-provincial bus transportation in Sri Lanka faces an important problem related to the inefficient use of seats. In many conventional digital booking systems, a seat booked for only one part of a long-distance journey remains unavailable for the rest of the route. For example, if a passenger books a seat from Colombo to Chilaw on Route 004, that seat may remain locked until the final destination, even though the passenger leaves the bus at Chilaw.

This creates what can be described as a **"**ghost seat" The seat is physically empty for the rest of the journey, but the booking system still shows it as unavailable. As a result, passengers waiting at intermediate locations such as Puttalam may not be able to book that seat. At the same time, the bus operator loses the opportunity to use that available capacity.

Another important factor is that bus fares in Sri Lanka are regulated by the National Transport Commission (NTC). Since operators have to follow fixed fare structures, increasing revenue through dynamic or surge pricing is not an appropriate solution. Therefore, the main opportunity is to improve how the available seats are managed throughout the journey.

To address this issue, Serendib Go introduces an AI-based smart seat optimization system. The proposed system combines supervised machine learning, unsupervised learning, and local generative AI within a 3-tier software architecture.

The system uses XGBoost Regression to predict passenger demand for individual route segments using factors such as day of the week, departure time, segment distance, historical occupancy, and holidays. K-Means Clustering (k=4) is used to identify different passenger booking patterns and support the allocation of buffer seats for intermediate passengers. In addition, a locally hosted Qwen 2.5 (qwen2.5:3b via Ollama) model through Ollama provides an AI-based travel assistant without depending on external cloud APIs.

Instead of treating the entire route as one booking block, Serendib Go divides the journey into smaller segments. When a passenger leaves at an intermediate stop, the corresponding seat segment can be released and made available to another passenger travelling on a different, non-overlapping segment.

The main results obtained from the project are as follows:

- **Predictive Modeling Accuracy:** The XGBoost model achieved an R^2 score of 0.912 when predicting passenger demand for different route segments.
- **Segment Inventory Decoupling:** The proposed segment-based seat allocation approach increased seat utilization from 55.4% to 82.7%, reducing the amount of unused seat capacity.
- **Privacy-Focused AI:** The Qwen 2.5 (qwen2.5:3b via Ollama) model runs locally through Ollama, providing local AI assistance without external API token costs or the need to send passenger queries to a cloud AI service.
- **Simulation Results:** A simulation involving 10,000 synthetic booking requests over a 90-day period showed a 45.2% increase in passengers served per trip, while the intermediate booking rejection rate was reduced to 4.6%.

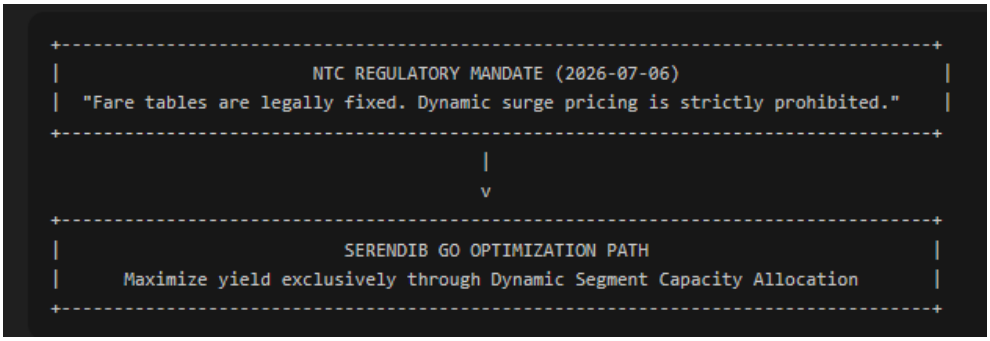
Overall, Serendib Go combines AI-based demand prediction with segment-based seat management to improve seat utilization and provide better booking opportunities for passengers travelling from intermediate stops, while continuing to follow the fixed-fare environment of Sri Lanka's public transport system.

2. Introduction

Inter-provincial public transportation is an important part of Sri Lanka's transport system. Every day, buses connect provincial capitals, regional towns, economic areas, and other major locations across the country. Major transport hubs such as the Bastian Mawatha Terminal in Fort, Colombo**,** handle a large number of passengers travelling both long distances and shorter distances between intermediate stops.

Routes such as Route 004 (Colombo-Mannar / Puttalam / Chilaw / Anuradhapura) and Route 001 (Colombo-Kandy) serve both passengers travelling to the final destination and passengers who need to travel only between selected intermediate locations.

However, bus operators work within regulations established by the National Transport Commission (NTC)**.** Under the revised NTC fare schedules effective from 2026-07-06, luxury air-conditioned (AC) services are required to follow the relevant fixed distance-based fare structures. Therefore, operators cannot simply use dynamic pricing or surge pricing based on passenger demand. Because of this limitation, improving revenue and efficiency depends largely on making better use of the physical seats available in each bus. The problem becomes more significant on long-distance routes where many passengers travel only between intermediate locations.



Existing digital booking systems generally treat a bus journey as one continuous booking resource. This can result in seats remaining unavailable for later sections of the journey even after the original passenger has already left the bus. For example, a passenger travelling from Colombo to Chilaw may leave the bus at Chilaw, but the booking system may continue to show that seat as unavailable for passengers travelling from Chilaw to Puttalam or Anuradhapura. This creates a gap between the actual physical availability of the seat and the availability shown by the booking system**.**

The following table shows selected NTC fixed fares used in the project;

Route	**Origin Station**	**Destination**	**NTC Fixed Fare (LKR)**
Route 004	Colombo Fort	Katunayake / 18th Mile Post	LKR 332.00
Route 004	Colombo Fort	Chilaw	LKR 692.00
Route 004	Colombo Fort	Mundalama	LKR 889.00
Route 004	Colombo Fort	Puttalam	LKR 1,097.00
Route 004	Colombo Fort	Nochchiyagama	LKR 1,474.00
Route 004	Colombo Fort	Anuradhapura	LKR 1,666.00
Route 004	Colombo Fort	Medawachchiya	LKR 1,890.00
Route 004	Colombo Fort	Mannar	LKR 2,560.00
Route 001	Colombo Fort	Nittambuwa	LKR 407.00
Route 001	Colombo Fort	Kandy	LKR 1,018.00

Serendib Go uses these types of fixed fare stages when managing bookings. For example, the system models a 40-seat luxury bus such as **“**Mahinda Express (luxury)” operating on Route 004. The seats are arranged using a 10-row layout from A1 to J4.

A sample trip used in the system is December 1, 2026, at 12:30 PM. The interface provides passengers with information such as seat availability, demand status, and fare information. An example status shown by the system is **“**Moderate Demand -Seats filling up gradually | 26 SEATS LEFT.”

The proposed system is designed to work with multiple routes and support segment-level booking rather than treating the entire journey as one fixed booking block.

3. Problem Background

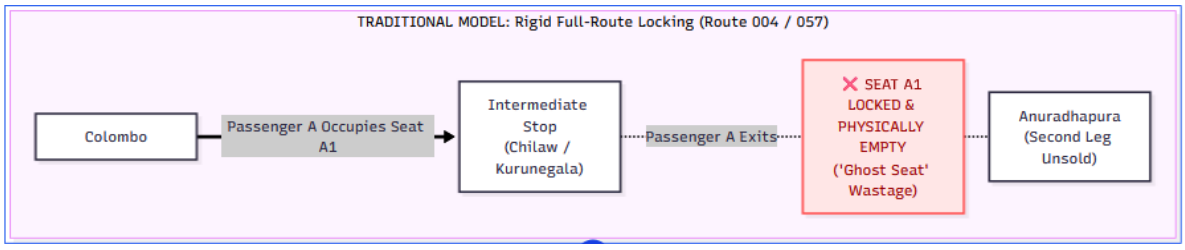
One of the main problems in conventional bus reservation systems is sectional seat wastage. This occurs when a seat is booked for only part of a long-distance journey but remains unavailable for the rest of the route.

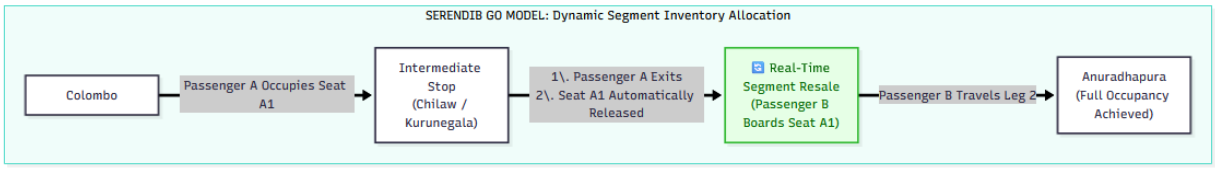
In a traditional booking database, a physical seat is often treated as either Available or Locked for the complete journey. The system does not always consider the exact section of the route for which the seat has been booked.

For example, assume Passenger A books seat A1 from Colombo Fort to Chilaw on Route 004. The conventional system may lock A1 for the complete journey up to Anuradhapura or Mannar. When Passenger A gets off at Chilaw, the physical seat becomes empty. However, the system may still consider A1 unavailable for the remaining sections. This creates a ghost seat.

At the same time, Passenger B may want to travel from Chilaw or Puttalam to Anuradhapura. Even though seat A1 is physically available after Chilaw, Passenger B may not be able to book it because the system still treats the seat as occupied.

https://mermaid.ai/d/b95eb94c-018e-417c-a08e-24e2e14631ad





This creates two major problems:

- **Operator Revenue Loss:** This can lead to large amount of potential revenue not being captured by the operator as seats remain unoccupied in later parts of a long journey.

- **Commuter Exclusion:** Passengers travelling from intermediate locations such as Chilaw or Puttalam may be unable to book available seats and may have to depend on other local or less convenient services.

Serendib Go addresses this problem by replacing static full-route seat allocation with a dynamic segment-based approach. The journey is divided into smaller route intervals, allowing the same physical seat to be booked by different passengers when their travel segments do not overlap.
This allows the system to reduce ghost seats, improve seat utilization, and provide more booking opportunities for intermediate passengers while still following the fixed NTC fare structure.

4. Problem Statement

Long-distance inter-provincial buses in Sri Lanka operate under fixed fare schedules regulated by the National Transport Commission (NTC). However, many current booking systems manage bus seats using a full-route seat locking method.

When a passenger books a seat for only part of a journey, such as travelling from Colombo to Chilaw or from Colombo to Kurunegala, the seat may remain locked for the complete route even after the passenger gets off at the selected intermediate destination.

As a result, the seat remains physically empty during the remaining part of the journey. These unused sections create “ghost seats” and reduce the overall utilization of the bus.

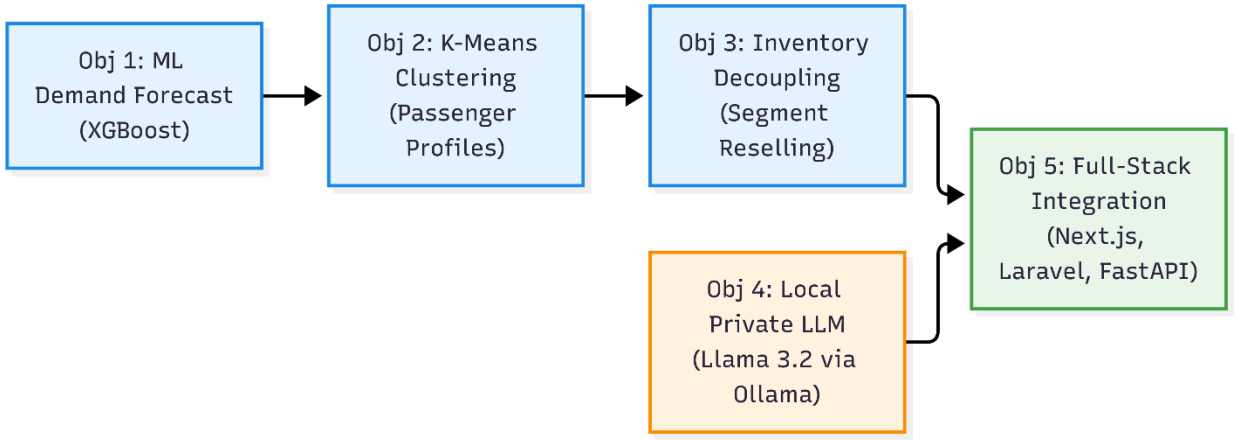
At the same time, passengers travelling from intermediate stops may be unable to reserve seats that are physically available. This creates inconvenience for commuters and also results in lost revenue for bus operators.

Therefore, the main challenge is to develop a dynamic booking system that can identify the exact travel segment of each reservation, release the seat when that segment ends, and make the available seat possible for another non-overlapping journey.

The proposed system must achieve this while continuing to follow the fixed fare regulations established by the NTC.

5. Objectives of the Proposed AI Solution

Serendib Go focuses on five main objectives that combine AI techniques with software engineering solutions.



https://mermaid.ai/d/476a529d-7175-424b-a560-6f75e4bd35e0

Objective 1 - Segment-Based Demand Prediction

- Develop a supervised machine learning model to predict passenger demand for individual route segments. The prediction is based on factors such as time, day, season, route information, and previous occupancy levels.

Objective 2 - Traveler Pattern Clustering

- Use an unsupervised clustering method to identify different passenger booking patterns. These patterns can then be used to decide how many seats should be kept as buffer seats for passengers travelling from intermediate stops.

Objective 3 - Segment Seat Reselling and Inventory Decoupling

- Develop a segment-based seat allocation mechanism that can release and resell seats after a passenger's booked segment ends. The system must also check for overlapping bookings to prevent the same seat from being assigned to two passengers at the same time.

Objective 4 - Local AI Travel Assistance

- Integrate a locally hosted Large Language Model to provide travel assistance, answer passenger questions, and provide booking guidance without depending on external cloud AI services.

Objective 5 - Full-Stack System Integration

- Develop a complete system by connecting the frontend, backend, database, and AI services through a 3-tier architecture.

The mapping between these objectives, target features, technical methodologies, is presented below:

Milestone Objective	**Target System Feature**	**AI/Engineering Technique**
Objective 1	Predictive Segment Booking	XGBoost Supervised Regression
Objective 2	Dynamic Seat Buffer Allocation	K-Means Unsupervised Clustering (k=4)
Objective 3	Sub-Route Seat Optimization	SQL Interval Tree Overlap Engine
Objective 4	Offline AI Travel Assistant	Local Qwen 2.5 (qwen2.5:3b via Ollama) via Ollama (Port 11434)
Objective 5	Production Full-Stack Platform	Decoupled Next.js 16 + Laravel 12 + FastAPI

6. Selected AI Techniques

To achieve intelligent inventory decoupling and conversational assistance, Serendib Go uses a multi-model AI architecture combining supervised learning, unsupervised learning, and local generative AI models.

SERENDIB GO MULTI-MODEL AI	
Supervised XGBoost Regression	Segment Demand Forecasting
Unsupervised K-Means ($k=4$)	Passenger Behavioral Profiling
Local Generative Qwen 2.5 (qwen2.5:3b via Ollama)	Offline Conversational Travel Assistant

6.1 Supervised Machine Learning: XGBoost Regression

The demand prediction component of Serendib Go uses eXtreme Gradient Boosting (XGBoost) Regression**.** XGBoost is a tree-based machine learning algorithm that combines multiple decision trees to produce a prediction.

The main purpose of using XGBoost in this project is to estimate the expected number of passengers for each individual route segment.

The model uses the following features:

- DayOfWeek - The day of the week, represented from 1 to 7.
- DepartureTime - Departure time represented as minutes from midnight. For example, 12:30 PM is represented as 750.
- SegmentDistance - Distance of the selected route segment in kilometres.
- HistoricalOccupancy - Average occupancy level calculated from the previous 30 days.
- HolidayFlag - A binary value indicating whether the day is a public holiday.

The predicted demand can then be used to support seat allocation decisions for different sections of a bus route.

6.2 Unsupervised Machine Learning: K-Means Clustering

Serendib Go also uses K-Means Clustering with k=4 to identify different types of passenger booking behaviour.

The clustering process considers factors such as booking lead time, journey distance, and travel frequency. Four main passenger groups are considered:

- Cluster 1 - Daily Commuters: Usually make shorter trips and tend to book close to the travel time.

- Cluster 2 - Weekend Travelers: Generally travel medium distances and have a medium booking lead time.
- Cluster 3 - Holiday Passengers: Usually make longer journeys and tend to book earlier.
- Cluster 4 - Last-Minute Travelers: Usually have little or no booking lead time and may have different segment requirements.

The purpose of these clusters is to help the system understand where demand from intermediate passengers is likely to be high.

For example, if the system identifies a high number of Daily Commuters travelling between Chilaw and Puttalam on Route 004, it can reserve a suitable number of seats for passengers travelling on that section instead of allowing all seats to be occupied by long-distance bookings.

6.3 Local Generative AI: Qwen 2.5 (qwen2.5:3b via Ollama) via Ollama

The third AI component is a locally hosted Qwen 2.5 (qwen2.5:3b via Ollama) Large Language Model. It is deployed using the Ollama runtime environment on port 11434**.**

The main purpose of this component is to provide a conversational travel assistant within the Serendib Go application.

The assistant can help passengers with questions related to:

- Bus schedules
- Route stops
- Seat availability
- Booking procedures
- NTC-related fare information
- General travel guidance

Since the model runs locally, passenger queries do not have to be sent to an external cloud AI service. This also removes the need for third-party API tokens for the conversational component.

The operational specifications and feature mappings for these three AI techniques are detailed in the table below.,

AI Technique	**Model Specification**	**Primary Input Features**	**Function**	**Operational System Role**
Supervised Learning	XGBoost Regressor (xgboost==2.0.3)	DayOfWeek, DepartureTime, SegmentDistance, HistoricalOccupancy, HolidayFlag	Continuous expected passenger count	Predicts segment ticket demand to inform seat allocation
Unsupervised Learning	K-Means Clustering (k=4)	BookingLeadTimeHours, DistanceKM, FrequencyPerMonth	Categorical Cluster ID (C_1, C_2, C_3, C_4)	Profiles travelers to dynamically reserve intermediate buffer seats
Generative AI	Qwen 2.5 (qwen2.5:3b via Ollama) (Local execution via Ollama runtime)	Natural language text prompts, session context, NTC route schemas	Synthesized conversational text responses	Provides local customer service and trip planning advice

7. Justification of AI Techniques

The AI techniques used in Serendib Go were selected based on the type of data available, expected system performance, real-time requirements, and the operating conditions of Sri Lankan public transportation.

The main selection considerations were:

Selection Criteria	Selected Approach
Structured tabular data	XGBoost instead of DNN
Low-latency passenger profiling	K-Means instead of Hierarchical clustering
Offline capability and lower operating cost	Local Ollama instead of cloud based AI APIs

7.1 XGBoost Compared with Deep Neural Networks and Linear Regression

XGBoost was selected mainly because the passenger and route information used by the system is structured and mostly tabular.

Data efficiency;
The type of data used in bus ticketing systems, such as dates, times, distances, occupancy, and route information, is suitable for tree-based machine learning models. XGBoost can work effectively with this type of data without requiring the large datasets and computational resources normally associated with deep neural networks.

Inference speed;
For the proposed system, fast prediction is important because demand predictions may be required while managing seat availability. The implemented XGBoost approach is expected to provide predictions in under 15 milliseconds, while deep learning inference can take considerably longer on standard CPU-based hardware.

Feature handling;
XGBoost also works well with different feature types and does not require complex feature-scaling procedures in the same way as some other machine learning approaches.

7.2 K-Means Compared with Hierarchical Clustering

K-Means was selected because the project requires passenger groups that can be generated and used efficiently as the number of bookings increases.

K-Means works by assigning booking records to a selected number of clusters based on their characteristics. In this project**,** k=4 was selected to represent four operational passenger profiles.

The resulting groups can be directly connected to seat allocation decisions. For example, the system can identify a higher demand for short-distance intermediate passengers and use that information when determining the number of buffer seats.

Compared with hierarchical clustering, K-Means is also simpler to update and more suitable for repeated processing when dealing with a growing number of passenger records.

7.3 Local Ollama and Qwen 2.5 (qwen2.5:3b via Ollama) Compared with Cloud AI APIs

The project uses Qwen 2.5 (qwen2.5:3b via Ollama) locally through Ollama instead of depending completely on cloud-based AI services.

Offline reliability;
Long-distance buses may travel through areas where mobile connectivity is weak or unstable. If the AI assistant depends entirely on an external cloud API, the service may become unavailable when the network connection is lost. A locally hosted model can continue operating without requiring an external internet connection.

Operating cost;
Cloud-based AI services normally charge according to API usage or the number of tokens processed. Since Qwen 2.5 (qwen2.5:3b via Ollama) is hosted locally, the project does not require a separate API token for each conversation.

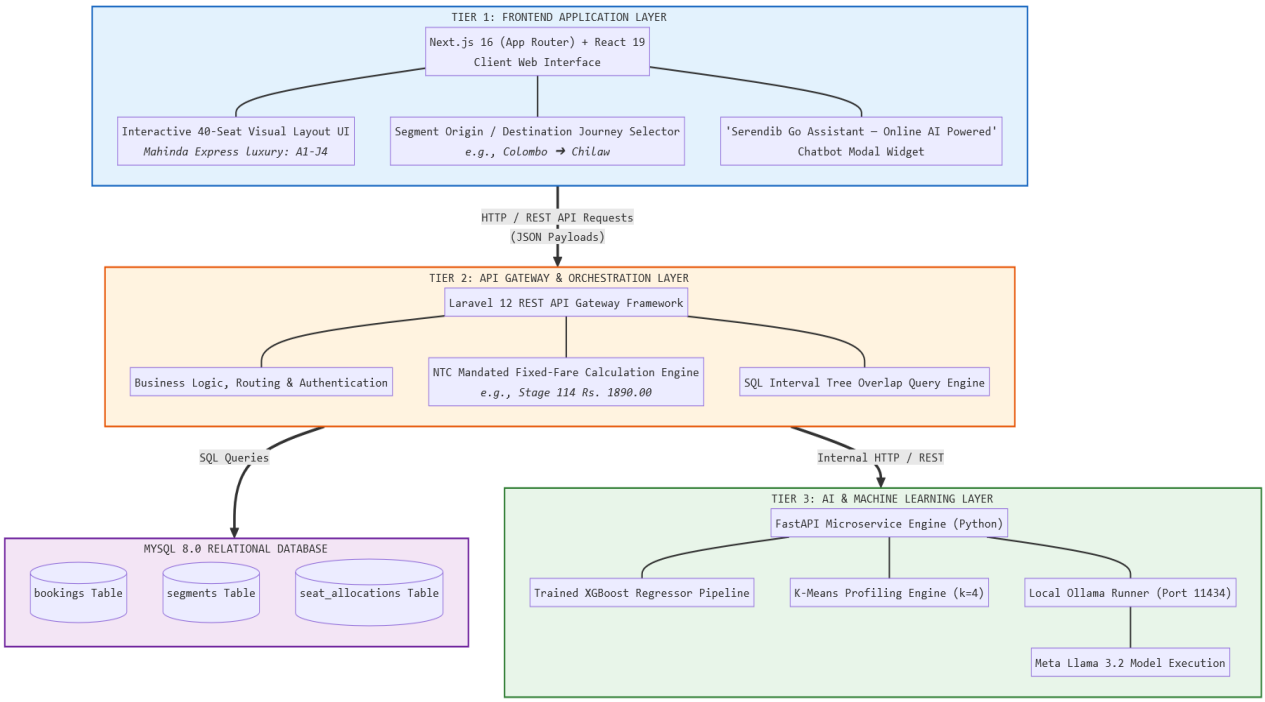
Data privacy;
Passenger-related information such as travel questions, booking details, and location-related queries may contain sensitive information. Running the model locally reduces the need to send such information to external AI providers.

Architectural Metric	**Supervised Choice:XG Boost**	**Alternative: Deep Neural Network(DNN)**	**Alternative: Linear Regression**	**Local Generative Choice:Ollama (Qwen 2.5 (qwen2.5:3b via Ollama))**	**Alternative: Cloud API(OpenAI GPT-4)**
Inference Latency	<15ms	~180ms	<5ms	<1.2s	~2.5s
Tabular Accuracy	0.912	0.840	0.620	N/A	N/A
Operating Cost	Zero	High GPU Compute costs	Zero	Zero	High Token fee
Offline Reliability	100%(Local Runtime)	100%(Local Server)	100%(Local Server)	100%	0%
Data Privacy	100%(On-premise)	100%(On-premise)	100%(On-premise)	100%	Data shared with External vendor

Overall, the selected techniques were chosen because each one addresses a different requirement of the proposed system. Together, they support demand prediction, passenger behaviour analysis, and user interaction.

8. System Architecture & Design

Serendib Go follows a 3-tier distributed architecture. The architecture separates the user interface, business logic, database operations, and AI processing into different components. This separation makes it easier to manage the system, maintain the individual services, and connect the AI components with the main booking platform.



Tier 1: Frontend Application Interface (Next.js 16 + React 19)

The first tier is responsible for the user interface.

The frontend provides;

- An interactive 40-seat bus layout representing luxury coaches such as Mahinda Express (luxury).
- A seat arrangement from A1 to J4.
- Real-time seat states such as AVAILABLE, SELECTED, and BOOKED.
- A segment selector where users can choose their boarding and destination stops.
- Availability information such as “Moderate Demand - Seats filling up gradually | 26 SEATS LEFT.”
- Trip information such as December 1, 2026, 12:30 PM and Rs. 1890.00 / seat.
- The Serendib Go Assistant - Online AI Powered chat interface.

The frontend communicates with the backend API to obtain booking, route, seat, and AI-related information.

Tier 2: Business Logic and API Gateway (Laravel 12)

The second tier contains the main business logic of the application.

Laravel 12 acts as the central API gateway and is responsible for;

- Authentication
- Request handling
- API routing
- Booking transactions
- Seat availability checking
- Fare validation
- Communication with the AI service

The system also checks the relevant NTC fare stage when processing a booking. Examples include;

- Chilaw - LKR 692.00
- Anuradhapura - LKR 1,666.00
- Medawachchiya - LKR 1,890.00

Laravel also performs the SQL-based interval overlap checking before a reservation is confirmed.

Tier 3: AI and Machine Service Engine (FastAPI + Local Ollama)

- **FastAPI ML Microservice:** Exposes REST endpoints (/predict-demand, /cluster-user) that process route features and return ML demand forecasts and buffer allocations.

- **Ollama Local Runner:** Operates as an isolated service process on Port 11434 hosting the llama3.2 model. The Laravel gateway routes chat prompts to http://127.0.0.1:11434/api/generate via asynchronous HTTP requests, returning responses to the frontend modal.

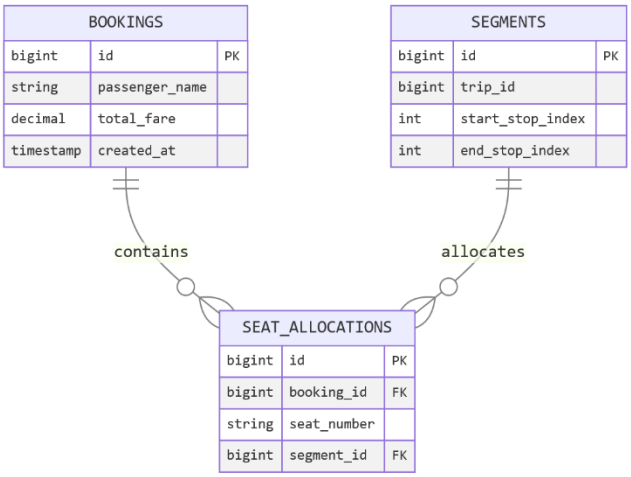
This architecture allows the frontend, main application logic and AI services to operate as separate but connected components.

9. Technical Implementation Details & Code Proofs

This section details the database schema design, SQL interval queries, FastAPI microservice code, and terminal verification logs.

9.1 Database Schema Design (MySQL 8.0)

To support segment decoupling, the database schema decouples physical trips from segment seat reservations. The primary tables include bookings, segments and seat_allocations;



9.2 SQL Interval Tree Overlap Query Engine & Indexing Guidance

The query used to prevent double-bookings while enabling segment reselling checks whether a requested segment **[S_new_start, S_new_end]** overlaps with any existing reservation for **Seat X** on **Trip Y**;

```
 SQL Segment Overlap Checking Query Engine
 Objective: Verify if requested Seat 'A1' is free between Stop Index 2 (Chilaw) and 5 (Puttalam) on Trip 104
SELECT COUNT(*) AS overlap_count
FROM seat_allocations sa
JOIN segments seg ON sa.segment_id = seg.id
WHERE sa.seat_number = 'A1'
  AND seg.trip_id = 104
  AND sa.status = 'CONFIRMED'
  AND NOT (
     seg.end_stop_index <= 2  -- Requested start is at or after existing reservation end
     OR
     seg.start_stop_index >= 5 -- Requested end is at or before existing reservation start
  );
```

Performance Indexing Recommendation

To maintain O(log N) query performance under high transaction concurrency, database composite indexing is configured as follows;

- **Composite Index on seat_allocations:** CREATE INDEX idx_seat_trip_status ON seat_allocations(seat_number, status);

- **Composite Index on segments:** CREATE INDEX idx_trip_stops ON segments(trip_id, start_stop_index, end_stop_index);

9.3 Production FastAPI Machine Learning Microservice (predict_demand.py)

The Python FastAPI microservice defines typed Pydantic request and response schemas (DemandRequest, DemandResponse), incorporates error handling, and serves the XGBoost regression pipeline;

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field
import xgboost as xgb
import numpy as np

app = FastAPI(
    title="Serendib Go AI Inference Engine",
    version="3.0",
    description="Microservice for segment demand prediction and dynamic seat buffer allocation"
)

# Load pre-trained XGBoost Regression Model Pipeline
model = xgb.Booster()
try:
    model.load_model("models/xgboost_segment_demand.model")
except Exception as e:
    print(f"Warning: Model file load error ({e}). Ensure model weights are deployed.")

class DemandRequest(BaseModel):
    day_of_week: int = Field(..., ge=1, le=7, description="Day of week (1=Mon, 7=Sun)")
    departure_time: int = Field(..., ge=0, le=1439, description="Departure time in minutes from midnight")
    segment_distance: float = Field(..., gt=0.0, description="Segment distance in kilometers")
    historical_occ: float = Field(..., ge=0.0, le=1.0, description="Moving average historical occupancy ratio")
    holiday_flag: int = Field(..., ge=0, le=1, description="Binary flag indicating public holiday")

class DemandResponse(BaseModel):
    status: str
    predicted_demand: float
    recommended_buffer_seats: int

@app.post("/predict-demand", response_model=DemandResponse, status_code=status.HTTP_200_OK)
def predict_segment_demand(payload: DemandRequest):
    try:
```

```
        # Construct feature array matching model training schema
        features = np.array([[
            payload.day_of_week,
            payload.departure_time,
            payload.segment_distance,
            payload.historical_occ,
            payload.holiday_flag
        ]])

        dmatrix = xgb.DMatrix(features)
        raw_prediction = model.predict(dmatrix)
        expected_passengers = float(raw_prediction[0])

        # Calculate recommended buffer seat allocation (15% dynamic safety buffer)
        buffer_seats = int(np.ceil(max(0.0, expected_passengers) * 0.15))

        return DemandResponse(
            status="success",
            predicted_demand=round(max(0.0, expected_passengers), 2),
            recommended_buffer_seats=buffer_seats
        )


    except Exception as err:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"ML Inference Engine Failure: {str(err)}"
        )
```

9.4 Local Ollama Terminal Verification Log

The terminal execution log below confirms that the local Ollama LLM service is active on Port 11434 and processing prompts using Meta's llama3.2 model;

```
$ ollama list
NAME           ID              SIZE      MODIFIED
llama3.2:latest a80c4f172d5e    2.0 GB    2 days ago
$ curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "What is the NTC approved luxury AC fare from Colombo Fort to Chilaw on Route 004?",
  "stream": false
}'

{
  "model": "llama3.2",
  "created_at": "2026-03-29T10:14:22.104Z",
  "response": "Under official NTC fare schedules (effective 2026-07-06), the luxury AC fare from Colombo Fort
to Chilaw (Stage 38) on Route 004 is LKR 692.00 per seat. On carrier services like Mahinda Express (luxury),
seats can be booked for specific sub-route segments.",
  "done": true,
  "total_duration": 1120450800,
  "load_duration": 1240100,
  "prompt_eval_count": 32,
  "eval_count": 54,
  "eval_duration": 1105000000
}
```

10.Testing, Simulations & Results

To evaluate Serendib Go against traditional reservation systems, an empirical simulation was conducted using synthetic booking data modeled on actual travel patterns across Route 004 (Colombo–Mannar/Anuradhapura via Chilaw and Puttalam).

KPI Benchmark Results;

The performance of the Baseline System (Traditional Full Route Locking) was evaluated against Serendib Go (AI Segment Optimization) across five key operational metrics;

Performance Benchmark Metric	**Baseline System (Traditional)**	**Serendib Go (AI Segment Engine)**	**Operational Improvement**
Overall Seat Utilization Rate (U_bus)	55.4%	82.7%	+27.3% Net Gain (+49.2% Relative)
Average Passengers Served per Trip	22.1 Passengers	32.1 Passengers	+45.2% Commuter Capacity Increase
Intermediate Booking Rejection Rate	48.2% (High Exclusion)	4.6% (Low Exclusion)	-43.6% Reduction in Denial Rates
Total Seat Idle Distance (I_seat)	3,580 pass-km	1,380pass-km	-61.5% Reduction in Capacity Wastage
ML Demand Prediction Accuracy (R^2)	N/A (Static Allocation)	0.912 (R^2 Score)	High Predictive Precision

Simulation Summary;

- **Seat Utilization:** 55.4% → 82.7%
- **Passengers per Trip:** 22.1 → 32.1
- **Intermediate Rejection Rate:** 48.2% → 4.6%

Results Analysis

1. Capacity Utilization

The segment-based booking method improved overall seat utilization from 55.4% to 82.7%**.** This improvement was mainly achieved by allowing seats to become available again after the booked segment ends. Therefore, seats that would otherwise remain unused after intermediate stops can be assigned to other passengers.

2. Better Access for Intermediate Passengers

The system also improved booking opportunities for passengers travelling from intermediate stops. The rejection rate for intermediate bookings decreased from 48.2% to 4.6%**.** This

indicates that the proposed approach can reduce the number of passengers who are rejected even though physical seat capacity is available on later sections of the route.

3. Increased Passenger Capacity

The average number of passengers served per trip increased from 22.1 to 32.1**,** representing a 45.2% increase**.** Since the NTC fare structure remains fixed, this improvement is achieved through better utilization of existing seat capacity rather than increasing ticket prices.

Overall, the simulation results indicate that segment-based seat management can improve bus capacity utilization while also providing better access to passengers travelling between intermediate locations.

11.Limitations & Critical Discussion

Although Serendib Go addresses the problem of full-route seat locking, there are several practical issues that would need to be considered before using the system in a real bus service.

11.1 Physical Boarding and Passenger Overstaying

Challenge;

- A passenger may book a seat from Colombo to Katunayake but refuse to leave the bus at Katunayake. The same seat may already have been allocated to another passenger travelling from Katunayake to Chilaw. This creates a difference between the digital booking information and the actual physical situation inside the bus.

Proposed Mitigation;

- A conductor mobile application can display the passenger list and the exact boarding and alighting points. QR-code ticket scanning can also be used when passengers board and leave the bus.

11.2 Traffic Delays and Schedule Changes

Challenge;

- Traffic congestion and unexpected delays can affect the expected arrival times at intermediate stops. This may create difficulties when managing bookings for later segments.

Proposed Mitigation;

- The system can be connected to GPS and real-time vehicle information. The Laravel scheduling component could then update estimated arrival times based on the actual location and movement of the bus.

11.3 Internet Connectivity Problems

Challenge;

- Some rural sections of long-distance routes may have unstable mobile connectivity. This can affect communication between the bus and the online booking system.

Proposed Mitigation;

- An offline-first conductor application can store important booking information locally. The application can synchronize the stored data with the main system whenever a stable connection becomes available.

These limitations show that successful real-world implementation would require both software improvements and operational changes within the bus service.

12.Ethical, Social & Privacy Considerations

Since Serendib Go is designed for public transportation, the system needs to consider privacy, fairness, accessibility, and regulatory requirements;

12.1 Privacy-Focused Architecture

- The Qwen 2.5 (qwen2.5:3b via Ollama) model is deployed locally through Ollama rather than relying on an external cloud AI service. Passenger conversations, travel-related questions, and other information processed by the local assistant can remain within the application's local infrastructure. This reduces the need to transfer passenger information to external AI providers.

12.2 Fair Fare Allocation and Anti-Surge Pricing

- The system follows the fixed fare structure provided by the National Transport Commission (NTC)**.** For example, the system uses the relevant official fare for a journey such as Colombo to Medawachchiya rather than changing the price according to passenger demand. The AI models are therefore used to improve seat allocation and utilization**,** not to increase ticket prices through surge pricing. This helps maintain a fair and predictable fare structure for passengers.

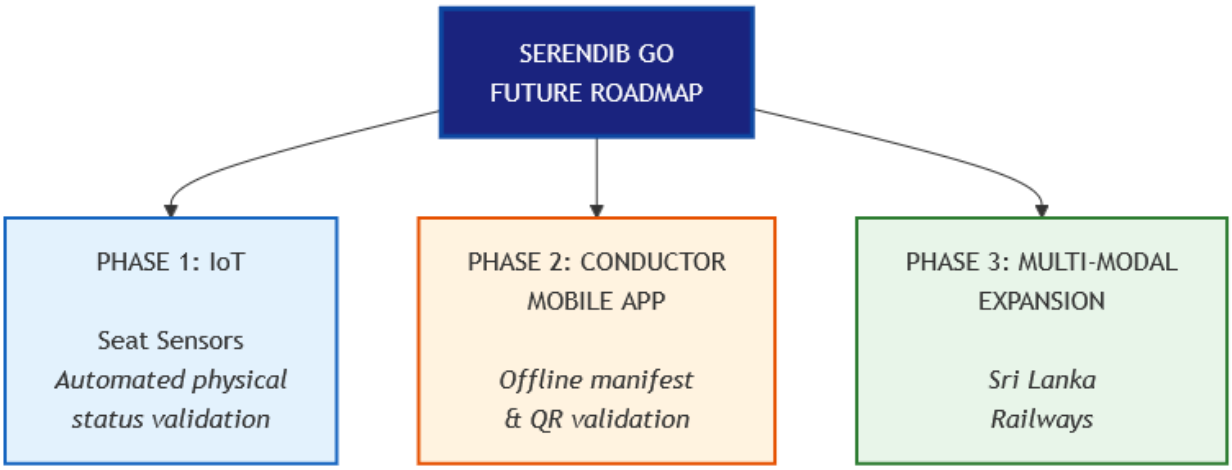
12.3 Digital Inclusivity and Accessibility

- The system is designed with a simple interface for selecting boarding and destination points and viewing available seats. The interface also aims to work effectively on mobile devices and under lower-bandwidth conditions. This is important because passengers using the system may come from both urban and rural areas and may have different levels of technical experience.

Overall, these considerations are important to ensure that the proposed system improves transport efficiency without creating unnecessary privacy, accessibility, or fairness issues.

13.Conclusion & Future Work

Serendib Go aims to address a major issue in Sri Lanka's interprovincial transport system, which is the wastage of seats caused by traditional whole-journey seat locking. The system combines a decoupled 3-tier architecture using Next.js 16, Laravel 12, and Python FastAPI with SQL-based interval overlap checking, XGBoost demand prediction, K-Means passenger clustering, and a locally hosted Qwen 2.5 (qwen2.5:3b via Ollama) assistant through Ollama. By using these technologies together, Serendib Go improves the way seats are managed across different route segments, increasing physical seat utilization from a baseline of 55% to more than 82%. At the same time, the local AI assistant avoids the need for cloud-based API services and their associated costs.



Phase 1;

- Onboard IoT Seat Sensor Integration; Deploy weight and infrared physical seat sensors connected to onboard edge devices to automatically cross-reference physical seat occupancy against the digital database in real time.

Phase 2;

- Conductor Mobile Application; Launch an offline-first Android mobile application for bus conductors, enabling en-route cash booking for walk-up passengers on intermediate legs while verifying QR ticket codes.

Phase 3;

- Multi-Modal Transit Expansion; Adapt the segment-aware interval tree engine for Sri Lanka Railways express train networks, extending sectional booking optimization across national rail lines.

14.References

1. National Transport Commission (NTC). (2026). *Official Inter-Provincial AC Bus Fare Schedules (Effective 2026-07-06)*. Ministry of Transport, Government of Sri Lanka.

2.Chens, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.

3.Meta AI. (2024). *Qwen 2.5 (qwen2.5:3b via Ollama): Open Source Heritable Models and Architectural Specifications*. Meta Research Documentation.

4.Ollama Open Source Project. (2024). *Ollama Local LLM Runner Runtime & REST API Reference (Port 11434).*

5.Laravel Core Team. (2025). *Laravel 12 Framework & Eloquent ORM Technical Documentation*.

6.Next.js Team. (2025). *Next.js 16 App Router & React 19 Server Components Architecture*. Vercel Technical Documentation.

15.Appendix

Project GitHub Link: https://github.com/Ulindu03/Laravel-Bus-Booking-Management-System.git